# Tema 2: Adquisición y transmisión de eventos: Apache Kafka.

Ángel Miguel García Vico
Rafael Pastor Vargas

amgarcia@scc.uned.es
rpastor@scc.uned.es

**Módulo 3: Gestión de datos en tiempo real**

**Infraestructuras Computaciones para Procesamiento de Datos Masivos**

2 de diciembre de 2022



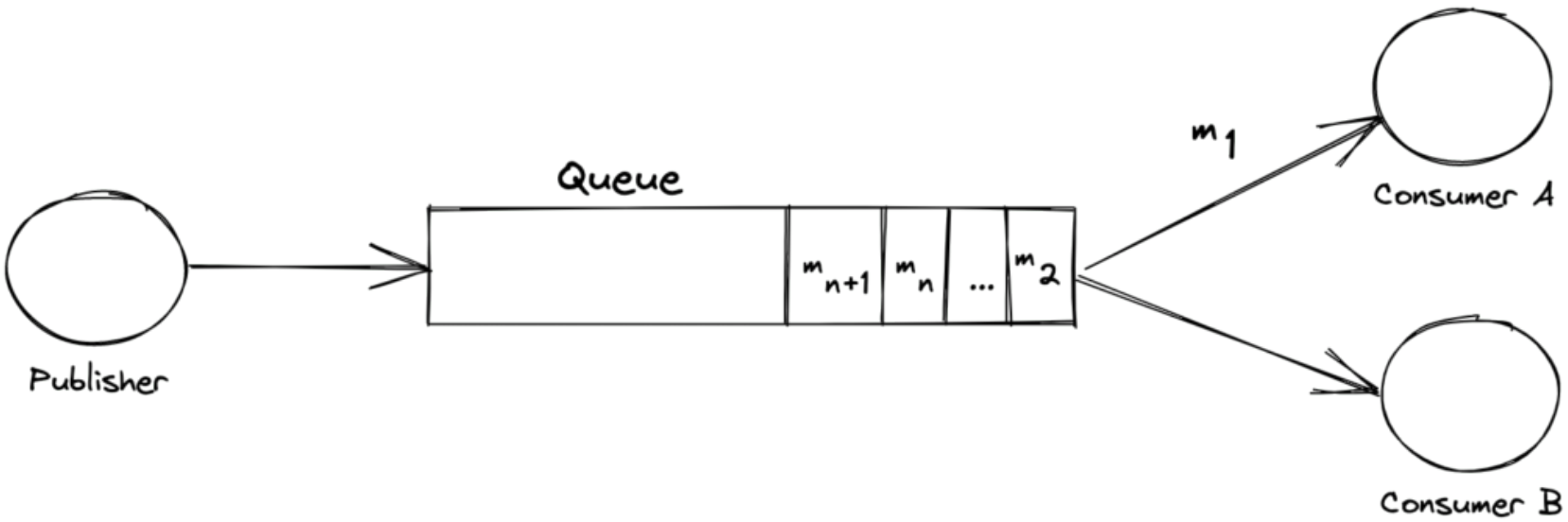
# Introducción: Motivación

Hemos visto en el tema anterior que es necesario que los datos lleguen de un stream (o varios) al sistema de procesamiento, que puede ser distribuido. ¿Cómo los hacemos llegar? A través de un protocolo de comunicación:

- En los paradigmas tradicionales, se requiere de una conexión directa entre dispositivos: el que genera datos y el que los procesa. Poco escalable.
- Se opta por otros paradigmas como los basados en colas de mensajes o los sistemas pub/sub que mejoran la escalabilidad de los sistemas.

# Introducción

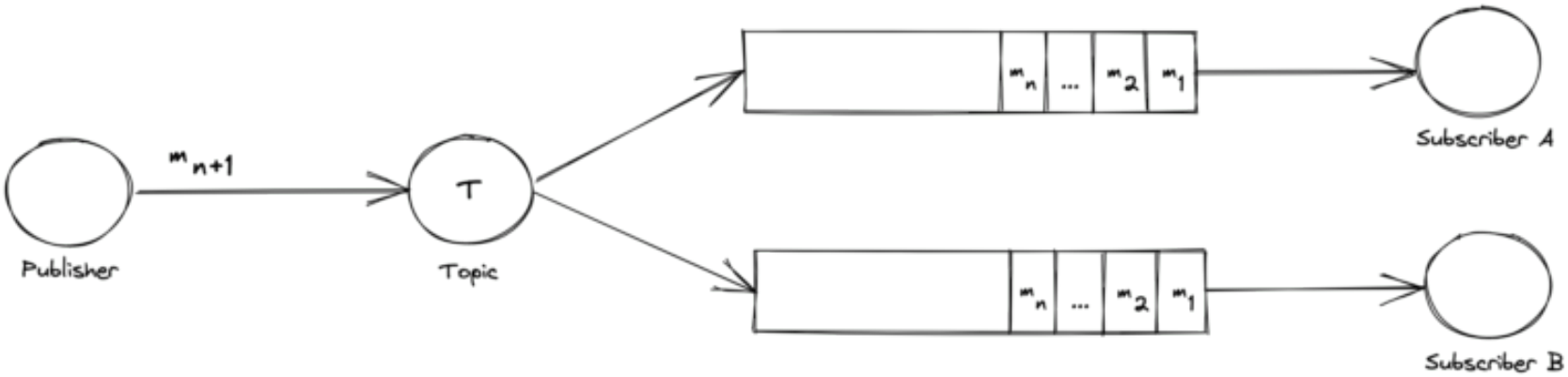
Sistema basado en colas de mensajes:




- El mensaje es consumido por **un único consumidor**.
- Uso común en micro-servicios. Útil para procesamiento masivo de tareas.
- Si falla un consumidor, el mensaje de la cola pasa a otro.

# Introducción

Sistema basado en pub/sub:




- Existe un nodo central que gestiona la/s cola/s de mensajes, llamadas topics. Dicho nodo es conocido como **bróker** y es el encargado del almacenamiento de los datos.
- Todos los consumidores suscritos a un topic reciben al menos una copia de los mensajes de la cola.
- Útil para distribuir datos a diferentes sistemas.
- **Tolerante a fallos:** si falla un consumidor, tras levantarse de nuevo puede recuperar aquellos mensajes que ha perdido.

# Introducción

Este tipo de sistemas funciona bien, pero cuando vamos a entorno de datos masivo surgen problemas:

- Rendimiento bajo (alta latencia, sobretodo).
- Almacenamiento de datos en memoria principal (no persistente y muy limitado)

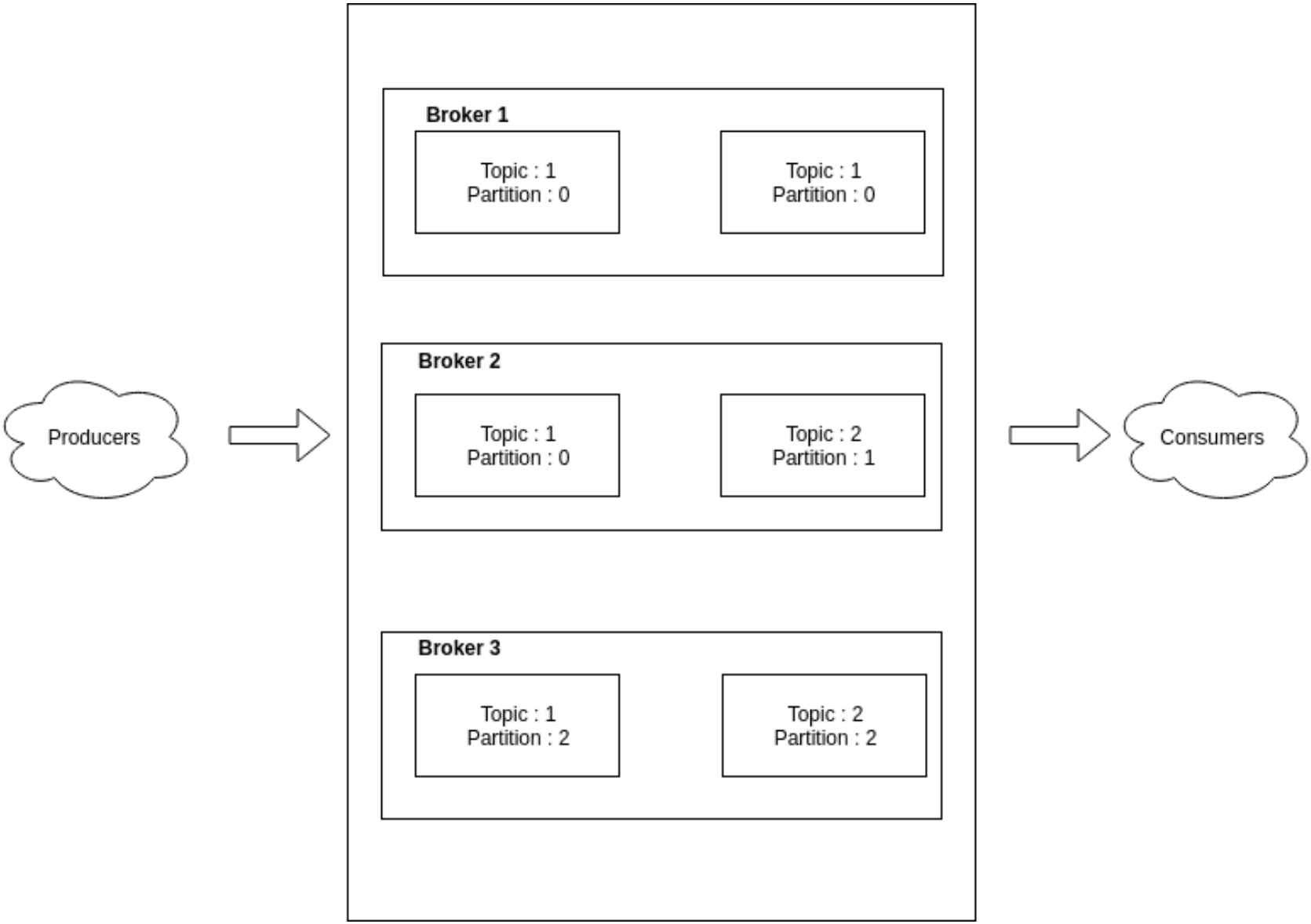
En entornos masivos, simplificar la entrega de mensajes, mejorar la latencia y el rendimiento, y que tenga un almacenamiento persistente o tolerante a fallos. Existen muchos sistemas actualmente:

- Google Cloud Pub/Sub
- Apache Pulsar
- Amazon Kinesis
- **Apache Kafka**

# Apache Kafka

- Kafka es una plataforma de streaming que sigue el patrón pub/sub para enviar mensajes/monitorizar eventos
- Alto rendimiento y baja latencia de transmisión de datos.
- Proporciona almacenamiento persistente de los datos de entrada durante el tiempo estipulado por el usuario.
- Proporciona un motor de procesamiento de Streams (Kafka Streams) para transformar los datos del topic antes de que sean enviado a los consumidores. No se considera en este curso.

# Arquitectura de Kafka



# Arquitectura de Kafka

Kafka posee una arquitectura muy distribuida y tolerante a fallos gracias a su alta replicación de datos:

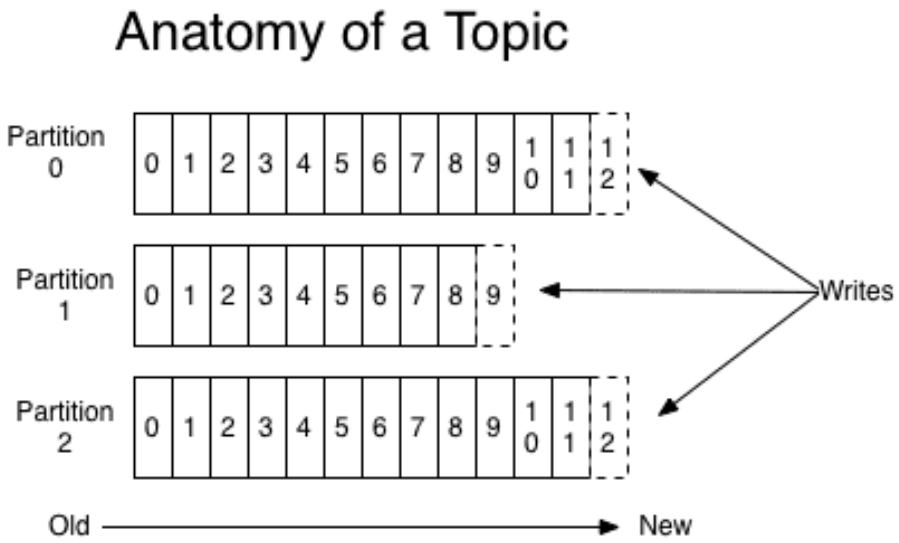
- Un cluster se compone de varios brokers:
  - Almacenan datos.
  - Normalmente, cada broker se encuentra en un servidor distinto.
- Cada broker, puede almacenar varios topics.
  - Un topic puede estar en varios brokers.
- Los datos de cada topic se almacenan en particiones:
  - Distintas particiones (o la misma, replicada) pueden estar en diferentes brokers.

# Arquitectura de Kafka: Zookeeper

ZooKeeper se utiliza para gestionar el descubrimiento de servicios y para hacer la elección de liderazgo de los brokers de Kafka. Envía los cambios de la topología a Kafka, de modo que cada nodo del clúster sabe cuándo se ha unido un nuevo broker, se ha caído, se ha eliminado un topic o se ha añadido otro, etc. Proporciona una vista sincronizada de la configuración del clúster de Kafka.

# Arquitectura de Kafka: Topics y particiones

Un topic se compone de varias particiones para mejorar la tolerancia a fallos y mejorar el throughput: una partición no es más que el propio flujo de datos, es la estructura de datos básica del sistema de Kafka.




- Se puede ver como un fichero log en el que los datos se **añaden al final**.
- Escritura/lectura **Secuencial**: mejora de rendimiento.
- Cada dato en cada partición tiene un identificador único llamado **offset**: útil para empezar a leer desde ese punto si el consumidor se cae.

# Arquitectura de Kafka: Topics y particiones

- Proporcionan escalabilidad
  - El tamaño del topic no depende de una sola máquina.
  - Se aumenta el throughput: se pueden servir datos en paralelo a varios consumidores.
- Proporcionan redundancia: al mantener varias copias de una misma partición (llamadas **replica**) en diferentes brokers, si uno se cae, la partición puede recuperarse.

# Arquitectura de Kafka: Topics y particiones

IMPORTANTE: Se garantiza el orden de llegada en los datos dentro de una misma partición, **pero no entre particiones**

# Arquitectura de Kafka: Replicación

- Una réplica es una copia de una partición. Están orientadas a la tolerancia a fallos.
- Factor de replicación: nº de copias realizadas.
- Se establece una réplica *leader*, que se encargará de recibir y mandar los datos a los consumidores. El resto de réplicas (*in-sync* o *follower*) se sincronizan con la *leader* de manera **asíncrona** a través de Zookeeper.

# Arquitectura de Kafka: Replicación

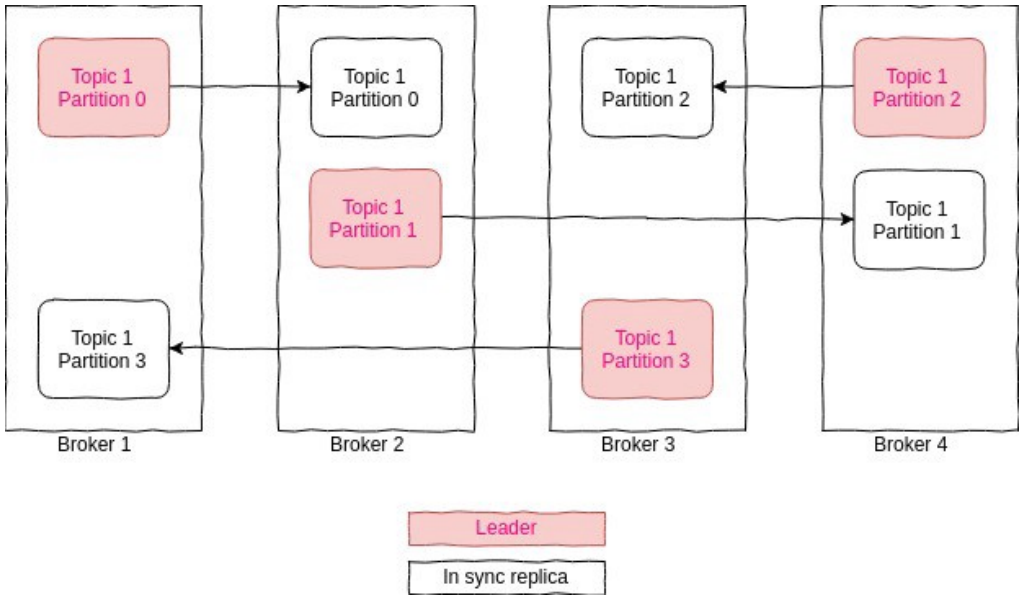



Figura: Cluster con 3 particiones y factor de replicación 2.

Problemas:

- Si el *leader* cae, puede haber pérdida de datos.
- Problemas con *followers*: pueden ser lentos, caerse, etc.

# Lectura y Escritura

- Escritura:
  - Los eventos o datos enviados por el productor se distribuyen entre las diferentes particiones. **Cada dato va a una única partición**.
  - De ahí que el orden de llegada de los datos se mantenga sólo dentro de las particiones.
  - Operaciones de escritura *append-only*: secuencial en disco.
  - Cada cliente puede leer desde el *offset* más antiguo hasta el más nuevo.

# Lectura y Escritura

- Lectura:
  - El consumidor puede elegir el *offset* desde el cual leer: debería mantener su último valor.
  - Kafka no almacena un estado del mensaje (quien lo ha leído): simplifica la lógica y soporta cualquier número de consumidores pero dificulta lógicas de entrega más complejas.
  - Habitualmente, se organizan en grupos: se asegura así que cada consumidor lee una partición diferentes y se aumente la escalabilidad.
  - Sólo puede haber un consumidor del grupo consumiendo una partición. Si hay más consumidores que particiones, los sobrantes quedan sin recibir datos, pero si uno de los otros se cae, éste recibe datos en su lugar.

# Resumen de la arquitectura

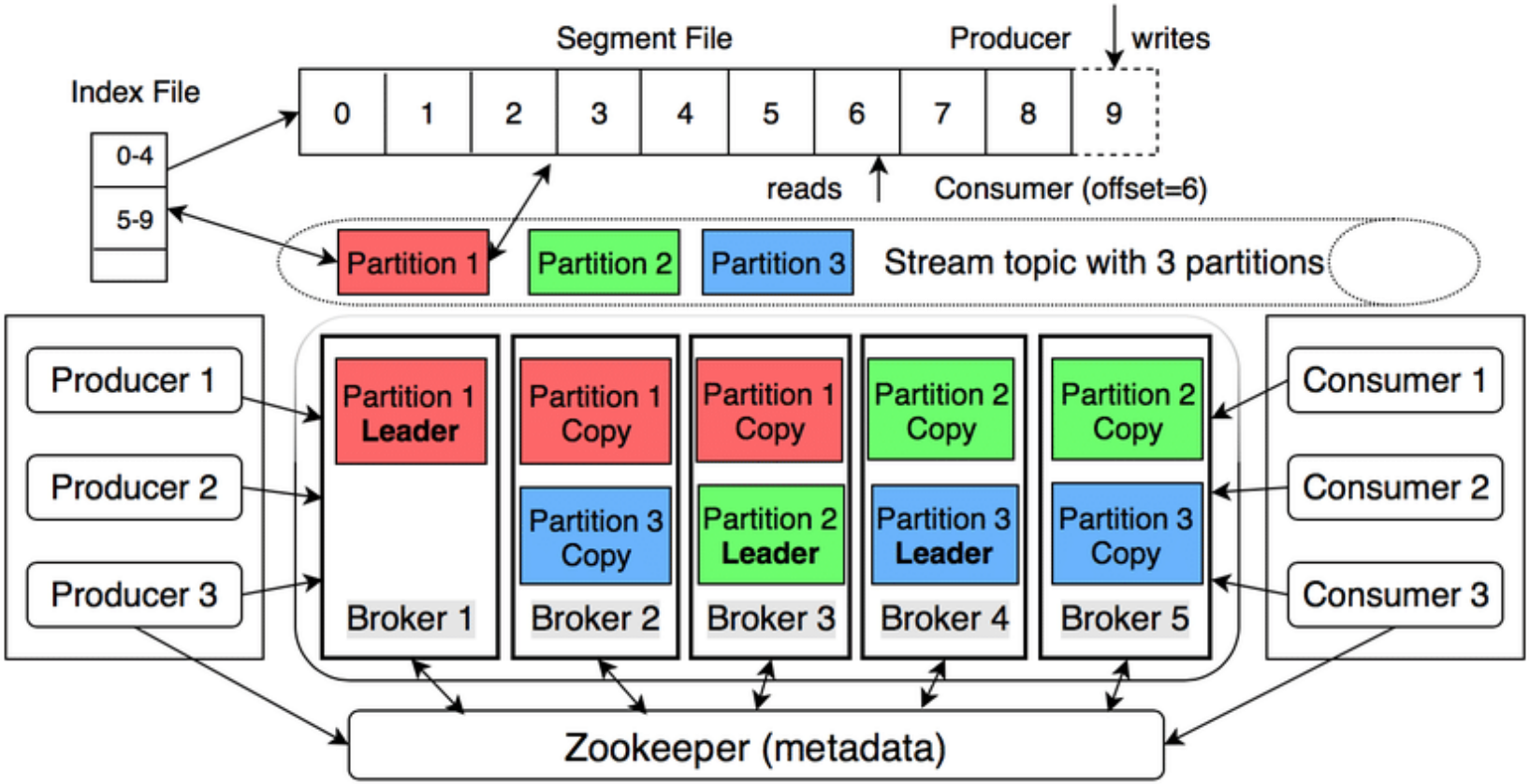



Figura: Ejemplo de Kafka con 3 Producers, 3 Consumers, 5 brokers, 1 topic, 3 particiones con factor de replicación 3.

# Kafka CLI (Command Line Interface)

Pasos a seguir....

1. Inicializar Zookeper.
2. Inicializar uno o varios brokers.
3. Crear topics.
4. Publicar datos a un topic.
5. Consumir datos del topic.

# Kafka CLI (Command Line Interface)

Inicializar Zookeeper:

- Definición de sus características en `config/zookeeper.properties`
  - Puerto (*clientPort*, por defecto 2181).
  - Almacenamiento de *snapshots* (*dataDir*)
- Comando de inicialización:
  - `zookeper-server-start.sh config/zookeeper.properties`

# Kafka CLI (Command Line Interface)

Inicializar Brokers:

- Definición de sus características en `config/server.properties`
  - Identificador (*broker.id*).
  - Otros parámetros
- Comando de inicialización:
  - `kafka-server-start.sh config/server.properties`

# Kafka CLI (Command Line Interface)

Creación/eliminación/descripción de topics

- Hay que indicar el cluster de brokers al que conectarse a través de la opción `--bootstrap-server` (por defecto `localhost:9092`)
- Comandos:
  - Crear topic: `kafka-topics.sh --create --zookeeper dir --replication-factor X --partitions Y --topic topic-name`. Donde *dir* es la dirección:puerto de Zookeeper, *X* es el número de replicas, *Y* es el número de particiones y *topic-name* es el nombre del topic.
  - Describir: `kafka-topics.sh --describe topic-name`
  - Listar topics: `kafka-topics.sh --list`
  - Borrar: `kafka-topics.sh --delete topic-name`

# Kafka CLI (Command Line Interface)

Publicación en un topic

- Hay que indicar el cluster de brokers al que conectarse a través de la opción `--bootstrap-server` (por defecto `localhost:9092`)
- Comando:
  - `kafka-console-producer.sh topic-name`
- Uso interactivo o a través de tuberías (*pipes*).

# Kafka CLI (Command Line Interface)

Consumo de un topic

- Hay que indicar el cluster de brokers al que conectarse a través de la opción `--bootstrap-server` (por defecto `localhost:9092`)
- Comando:
  - `kafka-console-consumer.sh topic-name`
- Algunas opciones interesantes del consumer:
  - `--from-beginning`. Se lee todo el topic (offset 0).
  - `--max-messages`. Máximo de mensajes a consumir antes de finalizar.
  - `--partition`. Consumir datos de una partición concreta.

# Lectura Recomendada

- Neha Narkhede, Gwen Shapira, Todd Palino. *Kafka. The definitive guide*. 2017. O'Reilly.
- Página oficial de Apache Kafka: `https://kafka.apache.org`